Booking Tool

3-Hour Hackathon MVP

Hackathon Team

Internal Equipment Reservation Tool

April 9, 2026

Problem

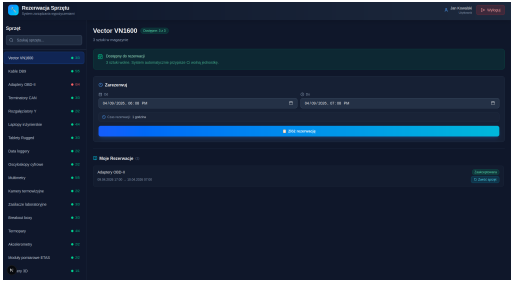
- Engineering and lab teams need a fast way to reserve shared equipment.
- Existing process is typically manual: chat, spreadsheet, or verbal coordination.
- This creates conflicts, poor visibility, and missing return tracking.
- Goal for the hackathon: deliver a usable MVP in 3 hours, not a production platform.

Hackathon Scope

- Login with seeded users and one admin account.
- Browse equipment by category with live availability counts.
- Create reservation requests for a selected time window.
- Auto-assign a free physical unit from the chosen category.
- Admin workflow: approve, reject, confirm return, add equipment units.
- File-based persistence for deterministic local demos and tests.

Architecture

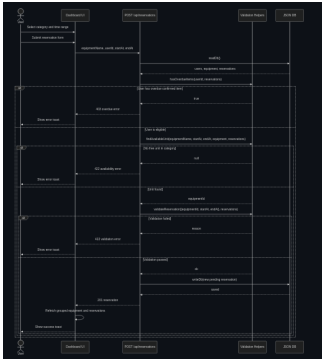
- Single Next.js application with client UI and route handlers.
- React frontend fetches data from internal API endpoints.
- Domain logic is centralized in validation and DB helper modules.
- Persistence uses a JSON file seeded from a template.
- Design choice was driven by hackathon constraints: minimal setup, fast iteration, easy reset.



Flow: Browser → Next.js UI → API routes → validation → JSON DB

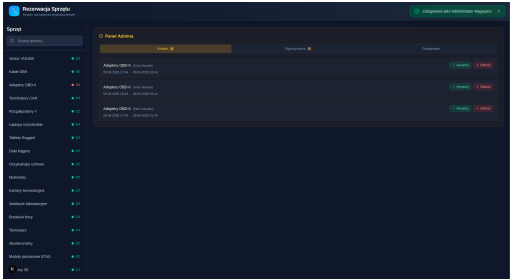
Key User Flow

1. User logs in.
2. User selects an equipment category.
3. User submits start and end time.
4. Backend checks overdue items and slot conflicts.
5. Backend finds a free unit in that category.
6. Reservation is stored with `pending` status.
7. Admin later approves or rejects the request.



What The MVP Already Handles

- Grouped equipment view with computed availability.
- Overlap detection per physical equipment unit.
- Blocking new reservations for users with overdue items.
- Reservation lifecycle: pending, confirmed, rejected, returned.
- Admin queue and active loan management from the same dashboard.
- Resettable local database for repeatable demos and end-to-end tests.



Tradeoffs For A 3-Hour Project

- JSON file instead of a real database.
- No session or token-based authentication.
- Authorization is mostly enforced in the UI, not fully server-side.
- Single-app architecture instead of separated services.
- Optimized for clarity, speed, and demo readiness over scalability.

Testing And Demo Readiness

- Unit and API tests cover validation and route behavior.
- Playwright covers end-to-end booking scenarios.
- Seeded users and reset scripts make the demo reproducible.
- Documentation includes architecture, use cases, and sequence diagrams.
- The demo can be reset quickly, which matters in a short hackathon presentation.

Next Steps Beyond The Hackathon

- Add real authentication and server-side authorization.
- Move persistence from JSON files to a proper database.
- Add audit trail, reservation editing, and cancellation.
- Introduce notifications for approval, rejection, and overdue returns.
- Deploy with multi-user safety and concurrent write handling.

Summary

- The project delivers a credible internal booking MVP within 3 hours.
- The solution is intentionally simple but functionally complete for a demo.
- Core value: clear reservation flow, admin control, and repeatable local setup.